



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1

AED2

September 8, 2026

Algoritmos y Estructuras de Datos

laJavaneta

Integrante	LU	Correo electrónico
Tamara Argañaraz	1189/24	tamaradiana.ar@gmail.com
Fran	/	@
Marti	/	@
Fernández Fazio, Adrián Patricio	938/24	adrianfernandezfazio@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

<http://www.exactas.uba.ar>

enum Categoria ES {Bronce, Plata, Oro, Platino}

Vuelos ES seq($\mathbb{R}$)

Millas ES $\mathbb{R}$

Usuario ES $\mathbb{Z}$

MillasGanadas ES seq(struct $\langle$ millas : Millas, nombrePromo : string, factorPromo : $\mathbb{R}$ $\rangle$)

MillasCanjeadas ES seq(Millas)

MillasRecibidas ES seq(struct $\langle$ transferidor : Usuario, millas : Millas $\rangle$)

MillasTransferidas ES seq(struct $\langle$ receptor : Usuario, millas : Millas $\rangle$)

MillasDisponiblesUsuario ES seq(struct $\langle$ usuario : Usuario, millas : Millas $\rangle$)

Transacciones ES struct $\langle$ vuelos : Vuelos, millasGanadas : MillasGanadas, millasCanjeadas : MillasCanjeadas, millasRecibidas : MillasRecibidas, millasTransferidas : MillasTransferidas $\rangle$

Usuarios ES dicc(usuario : Usuario, transacciones : Transacciones)

Características ES struct $\langle$ factor : $\mathbb{R}$, tope : $\mathbb{R}$ $\rangle$

Promo ES struct $\langle$ nombre : string, factor : $\mathbb{R}$ $\rangle$

PromoConTope ES struct $\langle$ nombre : string, características : Características $\rangle$

Promos ES seq(Promo)

PromosConTope ES seq(PromoConTope)

TAD AirMiles {

obs usuarios : Usuarios

obs promos : Promos

obs promosConTope : PromosConTope

proc crearSistema() : AirMiles {

requiere { true }

asegura {

res.usuarios = {} $\wedge$

res.promos = $\langle$ $\langle$ nombre : 'NoPromo', factor : 1 $\rangle$ $\rangle$ $\wedge$

res.promosConTope = $\langle$ $\rangle$

}

}

proc registrarUsuario(inout aerolínea : AirMiles, in usuario : Usuario) {

requiere {

usuario $\notin$ aerolínea.usuarios $\wedge$ aerolínea = aerolínea0

}

asegura {

aerolínea.usuarios = setKey(aerolínea0.usuarios, usuario, $\langle$ vuelos : $\langle$ $\rangle$, millasGanadas : $\langle$ $\rangle$, millasCanjeadas : $\langle$ $\rangle$, millasRecibidas : $\langle$ $\rangle$, millasTransferidas : $\langle$ $\rangle$ $\rangle$ $\wedge$

aerolínea.promos = aerolínea0.promos $\wedge$

aerolínea.promosConTope = aerolínea0.promosConTope

}

}

```

proc registrarUsuarioConReferidor(inout aerolínea: AirMiles, ) {
    in usuario:Usuario,
    in referidor:Usuario

    requiere {
        aerolínea = aerolínea0  $\wedge$  usuario  $\notin$  aerolínea.usuarios  $\wedge$  referidor  $\in$  aerolínea.usuarios
    }
    asegura {
        aerolínea.usuarios = setKey(agregarUsuarioReferido(aerolínea0.usuarios, usuario),
            referidor,  $\langle$ vuelos : aerolínea0.usuarios[referidor].vuelos,
            millasGanadas : concat(aerolínea0.usuarios[referidor].millasGanadas,
             $\langle$ millas : 500, nombrePromo : 'NoPromo', factorPromo : 1 $\rangle$ ),
            millasCanjeadas : aerolínea0.usuarios[referidor].millasCanjeadas,
            millasRecibidas : aerolíneas0.usuarios[referidor].millasRecibidas,
            millasTransferidas : aerolínea0.usuarios[referidor].millasTransferidas $\rangle$ )  $\wedge$ 
            aerolínea.promos = aerolínea0.promos  $\wedge$ 
            aerolínea.promosConTope = aerolínea0.promosConTope
        }
    }
}

aux agregarUsuarioReferido(usuarios : Usuarios, usuario : Usuario) : Usuarios =
    setKey(usuarios, usuario,  $\langle$ vuelos :  $\langle$ ,
    millasGanadas :  $\langle$ millas : 500, nombrePromo : 'NoPromo', factorPromo : 1 $\rangle$ ,
    millasCanjeadas :  $\langle$ , millasRecibidas :  $\langle$ , millasTransferidas :  $\langle$  $\rangle$  $\rangle$ )

proc acumularMillas(inout aerolínea: AirMiles, ) {
    in usuario:Usuario,
    in distanciaVolada:Vuelos,
    in promo:Promo

    requiere {
        aerolínea = aerolínea0  $\wedge$  usuario  $\in$  aerolínea.usuarios  $\wedge$  promo  $\in$  aerolínea.promos  $\wedge$ 
        distanciaVolada > 0
    }
    asegura {
        aerolínea.usuarios = setKey(aerolínea0.usuarios, usuario,
             $\langle$ vuelos : concat(aerolínea0.usuarios[usuario].vuelos,  $\langle$ distanciaVolada $\rangle$ ),
            millasGanadas : concat(aerolínea0.usuarios[usuario].millasGanadas,
             $\langle$ agregarMillasGanadas(aerolínea0.usuarios[usuario], distanciaVolada, promo) $\rangle$ ,
            millasCanjeadas : aerolíneas0.usuarios[usuario].millasCanjeadas,
            millasRecibidas : aerolíneas0.usuarios[usuario].millasRecibidas,
            millasTransferidas : aerolíneas0.usuarios[usuario].millasTransferidas $\rangle$ )  $\wedge$ 
            aerolínea.promos = aerolínea0.promos  $\wedge$ 
            aerolínea.promosConTope = aerolínea0.promosConTope
        }
    }
}

```

```

aux agregarMillasGanadas(distanciaVolada : Vuelos, transacciones : Transacciones, promo :
Promo) : MillasGanadas =
  ⟨millas : calcularMillasAcumuladas(distanciaVolada, transacciones, promo),
    nombrePromo : promo.nombre, factorPromo : promo.factor⟩

aux calcularMillasAcumuladas(distanciaVolada : Vuelos, transacciones : Transacciones, promo :
Promo) : ℝ =
  distanciaVolada * factorCategoria(transacciones) * promo.factor

aux factorCategoria(transacciones : Transacciones) : ℝ =
  IfThenElse(esBronce(millasTotales(transacciones)), 1,
    IfThenElseFi(esPlata(millasTotales(transacciones)), 1.5,
      IfThenElseFi(esOro(millasTotales(transacciones)), 2, 0)))

pred esBronce(millasTotales : ℝ) {
  millasTotales < 10000
}

pred esPlata(millasTotales : ℝ) {
  10000 ≤ millasTotales < 50000
}

pred esOro(millasTotales : ℝ) {
  50000 ≤ millasTotales < 1000000
}

proc canjearMillas(inout airMiles, in id : ℤ, in millasACanjear : ℝ) {
  requiere {
    (id ∈ usuarios[n]) ∧ millasACanjear > 0 ∧ usuarios0 = usuarios ∧ promo0 =
    promo
  }
  airMiles.usuarios[id] += [“millasCanjeadas”, millasACanjear] ∧ (∀ n : ℤ)((0 ≤ n <
  |airMiles.usuarios0[id]|) ⇒ LairMiles.usuarios[id][n] = airMiles.usuarios0[id][n] ∧
  promo = promo0 ∧ (∀ i : ℤ)((i ∈ airMiles.usuarios) ∧ (i ≠ id) ⇒ LairMiles.usuarios[i] =
  airMiles.usuarios0[i])
}

proc rankingMillas(in aerolinea :, ∈ cantidad : ℤ) : seq<<> > {
  requiere {
    cantidad ≥ 0
  }
  asegura {
    ((|res| = cantidad) ∨ (|res| = |aerolinea.usuario| ∧ cantidad = 0) ∧ ((usuario :
    ℤ ∧ Lusuario ∈ aerolinea.usuarios) ⇒ (∃ n : ℤ)(m : ℤ)(0 ≤ n, m < |res| ⇒
    L(ifThenElse((n > m) ∧ (millasTotales(aerolinea.usuarios[n]) > millasTotales(aerolinea.usuarios[m])
    ))
  }
}

```